RAIDFLASH

El die entero, a quien más lo necesita.

Johann · octubre 2026 · v0.1

---

0. De dónde sale esto

Esto no nace en un laboratorio de NVIDIA ni en un paper de serving. Sale de una persona ordinaria, con más curiosidad que título, mirando una GPU cara mientras una sola persona esperaba la respuesta.

El mercado ya sabía paralelizar. Los bancos públicos lo gritan: cientos de streams, miles de tok/s agregados. La cifra que se celebra es throughput del cluster. La cifra que ve el humano es tokens de un cursor. Esas dos cifras viven en planetas distintos. RAIDFLASH es el intento de juntarlas.

La idea cabe en una frase:

> Si el engine ya abre N caminos concurrentes para N inquilinos, esos mismos caminos pueden ser las tiras de un trabajo. O, si hay varias personas, la capacidad completa del die se reparte hacia quien más compute pide ahora.

Nadie tenía que inventar un kernel nuevo para verlo. Había que mirar el banco y preguntar para quién corre esa concurrencia.

---

1. El principio

Un engine moderno (TensorFold, vLLM, SGLang) ya es una fábrica de streams. `--parallel 8` en esta caja, `--parallel 32` o `256` en una estación GB300. Esa palanca existe para mezclar inquilinos: que el GPU no se duerma mientras cada usuario recibe 30–50 tok/s “interactivos”.

RAIDFLASH usa la misma palanca como ocupación. El techo que importa es el del silicio, no el del inquilino.

```
k = min(PARALLEL, k_codo_fase)
```

`PARALLEL` es el número de asientos del motor. `k_codo` es hasta dónde sigue pagando abrir otro asiento. Prefill y decode tienen codos distintos. El mapa se mide en este engine, como una tabla EFI: RPM × carga → potencia.

Un usuario solo: se intenta dar el 100% del GPU a esa persona, hasta el codo. Varios usuarios: se reparte el compute máximo disponible ahora hacia quien más lo necesita. Nadie se queda en un cursor C1 mientras hay lanes vacías. Nadie se lleva lanes más allá del codo si otro trabajo las pagaría mejor.

Eso es repartición dinámica de compute máximo. No es fairness de cola. Es llenar el motor.

---

2. Lo que el mercado midió

Los números públicos tienen todos la misma forma. Un stream solo es pobre. Muchos streams juntos son ricos.

caja	1 stream	saturado	fuente
2× DGX GB300 Flash	353 tok/s	2721 @ 32 · 3256 @ 256	StorageReview, 2026-10-05

caja	1 stream	saturado	fuente
GLM-5.3 TP2 una torre	188	5018 @ 256	mismo
DeepSeek V4.1 Flash	441	5248 @ 256	mismo
InferenceX B300	~50 / usuario	34 534 tok/s / GPU	Dynamo SGLang FP4
2× GB10 TensorFold (este paper)	49.5 D	121.1 D @ k=8	EFI live, `delta==sum`

El 353 frente al 2721 no es magia de un kernel secreto. Son 32 caminos del mismo modelo. El banco se los da a 32 sesiones. RAIDFLASH se los daría a las tiras de un documento, o a las 2–3 personas que de verdad están delante de la caja.

En B300, cientos o miles de tok/s para una persona dejan de ser ciencia ficción: son el agregado que ya existe, apuntado a un trabajo. En GB10 el salto es más chico (memoria LPDDR5x, 278 GB/s) y hay que decirlo: 2.45× en decode, no 14×. El principio es el mismo. El codo se mide.

---

3. Compute y memoria, dos motores en un die

Prefill frío (P-cold) es compute. El GEMM de expertos llena los SMs con un solo camino largo. Abrir 8 caminos fríos no sube el agregado: se reparten el mismo horno.

Decode (D) es memoria. Cada camino relee pesos y KV. Varios caminos a la vez amortizan esa recarga. Por eso D sube con k hasta el codo.

Prefill caliente (P-hot) es KV. Un seed deja el documento en cache. Las hermanas heredan el prefijo (~99% cached). El GEMM de esas hermanas casi no corre. El wall del prefill se parece al de un seed; el trabajo útil se multiplica.

fase	qué limita	qué hacer con k
P-cold	SMs / GEMM	k=1 por usuario; más k no paga
P-hot	slots + cache	k hasta PARALLEL; hermanas del mismo documento
D	DDR / recarga de expertos	k hasta el codo de tok/s agregado

Dentro de un engine regular: tiras concurrentes de prefill y de decode. P y D comparten el mismo pool de lanes. El inyector elige k según la fase y según cuánta gente hay.

---

4. Cómo se parte un trabajo

Un documento largo no es un cursor. Es un seed más k secciones.

1 Seed (P). Se prellena el acta una vez, con decode drafted

(`max_tokens ≥ 16`) para que el keep de DFlash sea resumible. Un seed serial (mt=8) deja `drafter_end=-1` y las hermanas vuelven a 0% cache. Medido.

1 Hermanas (P-hot + D). k streams cuyo prompt es el seed más “escribe

solo la sección i”. El prefijo pega en cache. El decode de las k tiras corre a la vez.

1 Fusión. Un POST `/v1/raidflash/complete` (packer in-engine) devuelve

el texto unido y un `usage`. El cliente ve un trabajo. El GPU vio k caminos.

k no es un 8 sagrado. En este GB10, 8 es el cap del engine. En un B300 el mapa puede pedir 32, 64 o 120. Se mide. Se copia la ley, no el número.

---

5. Tabla EFI de este engine

Caja: 2×GB10, GLM-5.3-Flash EXL3, TensorFold `v0.6.1-raidflash`, `--parallel 8`, `PREFILL_ROWS=4096`, `FILL_ROWS=2048`, pool 1.87M tokens. Tokens de `usage` del engine, `delta==sum`. 16 de octubre 2026.

Decode

k	agg tok/s	vs 1 lane
1	49.5	1.00×
2	71.0	1.43×
4	81.1	1.64×
8	121.1	2.45×

k_codo_d = 8. Sigue pagando.

Prefill frío

k	agg prefill tok/s
1	1586
2	1604
4	1651
8	1629

k_codo_p_cold = 1. Plano. El horno ya estaba lleno.

Prefill caliente (seed + hermanas, mt=16)

k	cache min	vs hacer k prefills seriales
1	99.1%	1.53×
2	99.0%	2.30×
4	99.0%	3.57×
8	99.0%	5.94×

k_codo_p_hot = 8. El seed se paga una vez.

Job (seed + hermanas stripe 80, un POST)

k	wall s	agg tok/s	cache
1	6.19	15.5	98.8%
2	6.73	26.1	98.8%
4	8.42	39.9	98.8%
8	9.82	66.8	98.8%

El wall casi no crece: el seed domina. El agregado sube porque las hermanas decodifican juntas.

---

6. Un usuario, o el que más lo necesita

El inyector (`EfiMap.plan(n, fase)`) lee esa tabla.

gente	D / P-hot	P-cold
1	[8] — el die es suyo	[1]
3	[3, 3, 2] — die lleno, nadie en cero	[1, 1, 1]
10	ocho personas a 1 lane; dos esperan un asiento	igual

Con una persona delante de CAKE (clínica, laboratorio, un box en casa) el objetivo es 100% del GPU para esa persona. Con tres, el objetivo es el máximo tok/s del die, partido hacia los tres, no 103 tok/s para uno y cero para los otros.

Quien más lo necesita: el trabajo en P-hot con documento propio se lleva hermanas (su KV no sirve al vecino). El trabajo P-cold se lleva una lane. El decode se llena hasta el codo. Las lanes ociosas son un fallo. Las lanes más allá del codo, también.

---

7. Qué se tocó en el engine, y qué no

RAIDFLASH 0.1 es una capa de ocupación sobre un engine regular.

- Overlay quirúrgico TensorFold: `DepthPolicy.next` usa `self.most`; admit no pisa `fnc7` con turbo `fc7`; packer in-engine `App.run`. - Flag `/tmp/raidflash.on`. Sin flag, el engine es el de siempre. - Apache-2.0. TensorFold no se fork. Los 107 parches Mia (EXL3 TP2, RoCE, prefix, fill) siguen siendo el motor GLM. RAIDFLASH es la política de para quién corren las lanes.

Falsado en este stack (no reabrir): árbol especulativo T3, overlap P‖D a nivel request (decode −59%), `MULTI_WINDOW=64`, seed mt=8 (0% cache). MLA / Hydragen es otro producto (Dynamo).

Vecino publicado más cercano: IPPD (arXiv 2609.05707), decode intra-prompt con contexto común. RAIDFLASH junta prefix cache, tiras y ocupación para un usuario hasta el codo, y el inyector cuando hay más de uno.

---

8. Lo que esto pide al futuro

En GB10 el decode se acerca a 2.5× con 8 tiras. El prefill caliente se acerca a 6× contra k seeds seriales. Eso es honestidad de una caja chica.

En B300 / GB300 el mismo dibujo, con más asientos, es el salto de 353 a miles. RAIDFLASH no inventa HBM. Apunta el agregado que el banco ya publica hacia la persona, o hacia las personas, que están usando la máquina.

El mapa EFI se vuelve a medir cuando cambia el die, el modelo o el `--parallel`. El computador es el mismo:

```
llenar lanes que pagan
partirlas entre quienes están
dar de más a quien más compute pide
parar en el codo
```

CAKE es el producto que tiene que responderle a gente, no a inquilinos. RAIDFLASH es la capa para que esa respuesta use el chip que se pagó.

---

9. Cierre

Los métodos de este v0.1 son toscos a propósito: un seed, k hermanas, una tabla EFI, un POST. Arcaicos. Casi tontos. El kernel fino lo va a escribir la comunidad. El golpe no es la implementación. Es el permiso:

usa el GPU que ya tienes, al 100%.

Quien iba a comprar otra tarjeta para “ir más rápido” puede gozar primero la que ya pagó. Quien creía que el techo era 50 tok/s interactivos puede apuntar el agregado del banco a su propio trabajo. Eso es disrupción de mercado con silicio soberano: AI local, en casa, en la clínica, en el laboratorio, sin racionar el die para inquilinos imaginarios.

Amamos el AI soberano. Amamos el AI local.

CAKE.

Johann, 2026. Apache-2.0. Código: `raidflash-engine` v0.1.

Números: `versiones/EFI-MAP-GB10-2026-10-17.json`. Ley: `docs/OCCUPANCY.md`. Inyector: `docs/EFI.md`.